

Learning Which Item to Evict Inside an Adaptive Replacement Cache

Yash Laxman, R Ajay Kumar, Kritharth B Shetty, and Pooja J Naik

Department of Computer Science and Engineering,
Sahyadri College of Engineering & Management, Mangaluru, India

Abstract. Every cache has to decide which item to remove when it runs out of space. ARC balances recency against frequency, but once it has picked a list to shrink it always removes the oldest item in that list. This chapter presents TIDE, a design that keeps ARC exactly as it is and hands the final eviction choice to a compact feedforward neural network trained continually on live traffic. The network regresses, for each of at most six candidates, a scaled logarithm of the time until the item is requested again, the quantity Belady’s optimal rule ranks by. It is trained online by backpropagation and Adam from delayed and right-censored labels that the request stream produces on its own, replayed through an experience buffer and applied by a background thread, so that training never blocks a request. A shadow copy of ARC takes control back when the network falls behind. We compare TIDE with seven classic and three learned policies. On fifteen synthetic workloads it has the best mean normalised hit rate (0.368, against 0.305 for ARC and 0.296 for Cacheus) and gains up to 48.6 points where recency is misleading. On three real traces (block storage, key-value and CDN) it does not beat ARC, trailing it by 0.39 points on average, while still beating S3-FIFO and every learned baseline. A compiled implementation with asynchronous training cuts the 99th-percentile request latency from 133–219 μ s to 18–26 μ s.

Keywords: Deep learning · Neural networks · Online learning · Cache replacement · Adaptive replacement cache · Eviction policy

1 Introduction

A cache keeps a small number of items close at hand so that repeated requests can be served quickly. When the cache is full and a new item arrives, something has to go. The rule that makes this choice is the replacement policy, and it decides the hit rate: the share of requests the cache can answer without going to slower storage. Even a few points of hit rate can matter a great deal, because every miss costs a disk read, a database query or a network trip.

The best possible rule has been known for a long time. Belady showed that a cache should remove the item that will be needed furthest in the future [2]. Real systems cannot see the future, so they guess. LRU guesses that recently used items will be used again soon. LFU guesses that popular items will stay popular. Each guess holds on some workloads and fails on others.

Learned policies try to replace these fixed guesses with a model trained on real traffic [14, 10]. In practice they tend to throw away the heuristic they replace, they add cost to every decision, and they can do much worse than the old rule when the model is wrong. We take a narrower path. ARC [11] already does one thing well: it decides whether the recency side or the frequency side of the cache should shrink. It does a second thing poorly: inside the chosen side it always removes the oldest item. TIDE keeps the first decision and hands the second to a neural network.

The learning problem is the centre of this chapter. The network sees a handful of candidates, each described by twelve numbers, and must predict how far away each candidate’s next request is. It never sees a labelled dataset: the correct answer for each prediction arrives later, when the item is requested again, or never arrives, in which case it must be treated as a censored observation. We show that a deliberately compact network, one hidden layer of 64 rectified linear units, trained continually by backpropagation on these delayed labels and stabilised by an experience replay buffer, is enough to learn periodic and phase-dependent reuse patterns that no fixed heuristic can see.

This chapter makes four contributions. It frames eviction inside ARC as a regression problem over a small, fixed set of candidates. It presents the network, its input representation, its target and its online training procedure in full, together with a compiled implementation that trains asynchronously. It gives simple arguments for how the design should behave, including its cost and how far it can fall behind ARC. Finally, it tests those arguments against ten baselines on synthetic workloads and on real traces, and reports where the design helps and where it does not.

2 Related Work

2.1 Belady’s Rule

Belady’s rule removes the item whose next request is furthest away, and no other demand paging policy can have fewer misses on the same sequence [2]. It is used as a yardstick. A common way to report results is a normalised hit rate, where 0 means no better than LRU and 1 means as good as Belady’s rule. When requests are drawn at random from a fixed set of popularities, the best rule that does not know the future is simply to keep the most popular items [1].

2.2 Classic Policies

Table 1 lists the classic policies that matter most here. Each one relies on a single idea about the workload, and each has a case where that idea breaks.

2.3 How ARC Works

ARC splits the cache into two lists. T_1 holds items seen once recently and T_2 holds items seen at least twice. It also remembers the keys of recently removed

Table 1. Classic replacement policies.

Policy	Main signal	Where it struggles
LRU	Time since last use	Scans and loops longer than the cache
LFU	Number of uses	Holds on to items that are no longer popular
2Q [6]	Trial queue, then LRU	Queue sizes are fixed in advance
LIRS [5]	Gap between uses	Very small caches; extra bookkeeping
ARC [11]	Recency and frequency lists	Always removes the oldest item of a list
S3-FIFO [18]	Small and main FIFO queues	No view of when an item returns
SIEVE [20]	FIFO with a visited bit	Only a yes or no view of reuse

items in two ghost lists, B_1 and B_2 , without keeping their data. If a request hits a ghost list, ARC takes it as a sign that it shrank the wrong list, and it moves a target size p in the other direction. When space is needed, ARC removes an item from T_1 if T_1 is larger than p and from T_2 otherwise.

This choice of list is ARC’s strength. Its weakness is what happens next: it always removes the least recently used item of that list. On a loop that is slightly longer than the cache, the oldest item is exactly the one that will be needed next, so ARC gets no hits at all.

2.4 Learned Policies

Several groups have trained models to make eviction decisions (Table 2). Hawk-eye learns from Belady’s rule replayed on past requests [4]. LeCaR weighs an LRU and an LFU expert by the regret of their past evictions [16], and CACHEUS extends it with scan- and churn-resistant experts [13]. LRB trains gradient-boosted trees online for content delivery caches [14]. Parrot imitates Belady’s rule with a deep network trained offline, and reports that imitation outperforms a reinforcement learning formulation of the same task [10]. HALP is the closest to our idea: a heuristic suggests a few candidates and a small neural network with one hidden layer picks one [15]. None of them uses ARC’s choice of list as a starting point.

3 Problem Formulation

We want eviction choices that are closer to Belady’s rule without giving up what ARC already does well. We also want the extra cost of each decision to stay small and fixed, and we want the design to fall back to ARC when the model is wrong.

The key quantity is the reuse distance d of an item: how many requests from now it will be asked for again. Belady’s rule removes the item with the largest d .

Table 2. Learned replacement policies.

Policy	Learns from	Role of a heuristic
Hawkeye	Belady’s rule on past requests	None
LeCaR	Regret on removed items	Mixes LRU and LFU
CACHEUS	Regret on removed items	Mixes two heuristics
LRB	Relaxed Belady rule, online	None
Parrot	Belady’s rule, offline	None
HALP	Preference feedback, online	Suggests candidates
TIDE	Time until next request, online	ARC picks the list and is the fallback

We do not need the exact value, only the order of the candidates, so the network predicts

$$g(d) = \log \left(1 + \frac{d}{c} \right), \quad (1)$$

where c is the cache size. Dividing by c puts caches of different sizes on the same scale, and the logarithm stops a few very large distances from dominating the squared-error loss. Because g is monotone, ranking candidates by $g(d)$ is the same as ranking them by d , so a perfect regressor reproduces Belady’s choice among the candidates it is shown.

We treat this as regression rather than classification on purpose. A classifier that predicts whether an item will be hit if kept would need a calibrated probability for each candidate and a further rule to rank them. A regressor on $g(d)$ gives a ranking directly, and its target is exactly the quantity the optimal offline rule uses.

4 Proposed Method

4.1 Overview

TIDE keeps all of ARC: its four lists, its target size and its choice of which list to shrink. The only change is the last step. Instead of removing the oldest item of the chosen list, TIDE scores a few candidates with the network and removes the one it expects to be needed last. Figure 1 shows the parts of the design: a decision path that runs on every miss, and a learning path, run by a separate thread, that turns what happens next into training data.

4.2 Candidates

For the list ARC has chosen, TIDE looks at up to six items:

- the four oldest items, where the oldest one is ARC’s own choice;
- the newest item, which is often the right one to drop on long loops, because it will not come back until the loop comes round again;

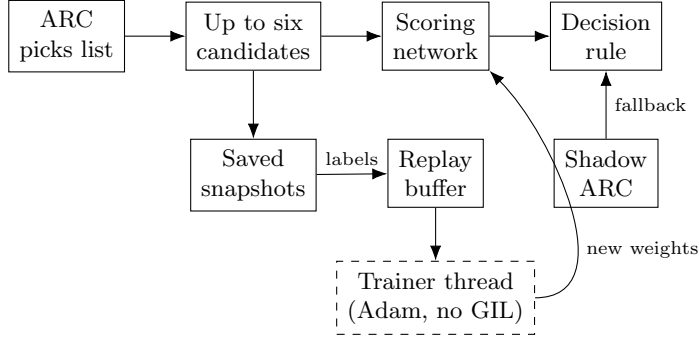


Fig. 1. The parts of TIDE. ARC picks the list, the network ranks a few candidates, snapshots of every scored candidate become training examples once their outcome is known, a background thread trains on them and publishes new weights, and a shadow copy of ARC decides when to hand control back.

- on a miss for a new item when the cache is full, the item being requested, which lets TIDE serve it without storing it. We call this a bypass. It protects the cache from one-time scans.

The number of candidates never depends on the cache size. Limiting the network to the tail of the list follows LRB and HALP, which found that a few cold candidates capture most of the benefit [14, 15].

4.3 Input Representation

TIDE keeps a short history for each item it has seen recently, including items that were removed, so it can recognise them when they come back. This history is capped at $8c$ entries, and items that are in the cache always keep theirs. From the history it builds the twelve numbers in Table 3, which form the input vector $x \in \mathbb{R}^{12}$. Every time value is measured in requests, divided by c and passed through $\log(1 + \cdot)$, the same transform as the target, so that the same input means the same thing whether the cache holds 16 items or 20,000. A gap that has not yet been observed, for an item seen fewer than four times, is filled with the sentinel $\log(1 + 2W/c)$ used for censored labels in Sect. 4.5. None of the features look at the content of an item, so the design works with any kind of key.

As an example, take $c = 100$ and an item last requested 40 requests ago, whose three previous gaps were 95, 105 and 98 requests. Its first four features are $\log 1.40 \approx 0.34$, $\log 1.95 \approx 0.67$, $\log 2.05 \approx 0.72$ and $\log 1.98 \approx 0.68$. Three nearly equal gaps tell the network that this item returns on a clock of about 100 requests, which a feature based on recency alone cannot express.

There is no feature for trend. An item that is gaining popularity already shows shorter gaps and a short term counter that is higher than its long term one, and the network can learn to read that combination.

Table 3. The twelve input features computed for each candidate. Time values are scaled as $\log(1 + t/c)$.

No.	Feature	What it captures
1	Time since last request	Recency
2 to 4	Last three gaps between requests	Regular patterns and loops
5	Smoothed average gap	Typical return time
6	Request count, capped at 3 and scaled to $[0, 1]$	Whether the item is new or repeat
7 to 9	Three decaying counters, half-lives $c, 4c, 16c$	Popularity over short, medium and long spans
10	$\log(1 + \text{total request count})$	Long term popularity
11	Is this the requested item?	Marks the bypass option
12	Is it in the frequency list?	ARC’s own view of the item

4.4 Neural Network Architecture

The scoring network is a multilayer perceptron with one hidden layer of 64 rectified linear units and a linear output (Fig. 2). For a candidate with input $x \in \mathbb{R}^{12}$ it computes

$$h = \text{ReLU}(W_1 x + b_1), \quad W_1 \in \mathbb{R}^{64 \times 12}, \quad b_1 \in \mathbb{R}^{64}, \quad (2)$$

$$\hat{y} = w_2^\top h + b_2, \quad w_2 \in \mathbb{R}^{64}, \quad b_2 \in \mathbb{R}, \quad (3)$$

where $\hat{y}$ is the predicted $g(d)$. The network has $12 \cdot 64 + 64 + 64 + 1 = 897$ trainable parameters. All candidates of a decision are scored together as one batch of at most six rows, and the victim is chosen from the six outputs by the decision rule of Sect. 4.6.

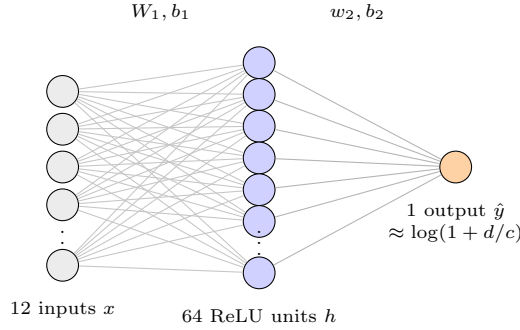


Fig. 2. The scoring network: twelve features, one hidden layer of 64 rectified linear units, and a linear output that predicts the scaled reuse distance.

Initialisation and optimisation. W_1 is drawn from a zero-mean normal distribution with variance $2/12$, the He initialisation for rectified units [3], and w_2 with variance $1/64$. Biases start at zero, except that just before the first gradient step the output bias b_2 is set to the mean of the labels collected so far. The network therefore starts by predicting the average reuse distance and only has to learn deviations from it, which removes a large initial transient from the loss. Training minimises the mean squared error between $\hat{y}$ and $g(d)$ with Adam [7] at a learning rate of 10^{-3} and $\beta_1 = 0.9$, $\beta_2 = 0.999$. Adam’s per-parameter step sizes matter here because the data stream is not stationary: when the workload changes phase, the distribution of both inputs and targets shifts, and an adaptive optimiser recovers faster than a fixed step.

Why a small network. The network is small by design, for three reasons. First, it runs on every miss, inside a latency budget of microseconds, and it only ever ranks at most six candidates, so extra capacity would add cost on every decision. Second, the features of Table 3 already summarise the part of an item’s history that matters for its next reuse: its recent gaps, their smoothed average and its popularity at three time scales. A sequence model such as a recurrent or temporal convolutional network would have to learn this summary from raw request histories at a much higher cost per decision, and we have not found a reason to expect it to rank six candidates better. Third, the quality of the labels, not the expressiveness of the model, is the main limit on what can be learned online. The same conclusion was reached for HALP, which uses a network of the same shape in production [15]. What makes TIDE a learning system is therefore not the depth of its network but the way it is trained, described next.

4.5 Online Training from Delayed Labels

The network is trained continually while the cache runs, with no offline dataset. At decision time the true reuse distance of a candidate is unknown by definition, so TIDE uses delayed labelling. Each time a candidate is scored, a snapshot of its feature vector and the current time t_0 is saved. If the item is requested again at time t_1 , the snapshot is labelled $g(t_1 - t_0)$. If it has not come back after a window of $W = 8c$ requests, the observation is right-censored: TIDE knows only that $d > W$. Dropping such examples would throw away the clearest evidence that an item was not worth keeping, so they are given the fixed label $\log(1 + 2W/c) = \log 17 \approx 2.83$, as if the item had returned after twice the window. Snapshots are taken for every scored candidate whatever the decision was, because the label depends only on when the item is requested next and not on what the cache did.

Labelled examples go into a circular replay buffer of 32,768 rows. After every 32 new labels, the network takes one Adam step on a minibatch of 256 rows drawn uniformly at random from the buffer. This is the experience replay mechanism introduced for reinforcement learning agents [9], used here for the same reason: sampling at random breaks the strong correlation between consecutive

requests and keeps gradient estimates stable. The finite buffer also acts as a sliding memory. Old examples are overwritten, so patterns from an earlier phase of the workload fade out of the training set as new ones arrive.

The network is not used to make decisions until at least W requests have been seen and at least 2,048 labels have been collected. Until then every decision is ARC’s, and the network only takes over once it has seen enough outcomes to have learned a meaningful mapping from features to reuse distance.

4.6 Decision Rule

TIDE removes the candidate with the highest score, with one condition: any candidate other than ARC’s own choice has to win by a margin. For the other old items the margin is 0.1 on the scale of g , which means the network must expect the item to return about 10% later, in units of $1 + d/c$, than ARC’s choice. For the newest item and the bypass option the margin is 0.2, or about 22% later, because these moves are riskier. When the network is unsure, ARC’s choice stands. The margin turns “is this candidate better” into “is this candidate clearly better”, which keeps a slightly noisy network from second-guessing good ARC decisions.

4.7 Fallback

A second copy of ARC, the shadow, sees the same requests and keeps only keys. TIDE counts its misses and the shadow’s misses over windows of $\max(2000, 4c)$ requests. If TIDE misses more than $\max(10, 1\%)$ of the window more than the shadow, it stops using the network and lets ARC decide for a while. The pause starts at two windows and doubles after each repeated failure, up to 32 windows. A window in which TIDE is at least as good as the shadow reduces the failure count again. Training carries on during the pause, because labels do not depend on who made the decision.

4.8 Implementation and Asynchronous Training

A first implementation in Python and NumPy spent most of its time outside the arithmetic: interpreter overhead in building feature rows, the fixed cost of calling into NumPy for a six-row batch, and an inline training step every 32 labels that stalled the request that triggered it. We rewrote the hot path with the Numba compiler [8], keeping the policy itself unchanged.

Metadata as arrays. Per-item history is stored in a preallocated array of nine floating-point values per item, with a missing gap marked as NaN, instead of one Python object per item. *One compiled call per decision.* A single compiled function builds the feature rows of all candidates, which double as the label snapshots, and scores them, replacing six feature calls and a library call. *Asynchronous training.* The request path only writes labels into the replay buffer and signals a background thread every 32 labels. The thread runs the Adam

steps in compiled code that releases Python’s global interpreter lock, on its own copy of the weights, and publishes each new copy with a single reference swap. Decisions therefore never wait for training and never see half-updated weights. The thread runs its whole backlog of steps in one compiled call: re-acquiring the interpreter lock after every step would cost up to one interpreter switch interval while the request thread is busy, and in a first version this starved the trainer, which completed less than a tenth of the intended steps.

A synchronous mode, which trains inline exactly when the original does, is kept for reproducible experiments. Its hit rates match the NumPy reference within 0.3 points in our tests, and its compiled Adam step matches the NumPy step to within 10^{-7} . Table 4 lists all settings. They were fixed before testing and are the same for every workload.

Table 4. Design settings, fixed in advance for all workloads.

Setting	Value	Reason
Candidates per decision	Up to 6	Keeps cost fixed
Network	$12 \rightarrow 64 \rightarrow 1$, ReLU	897 parameters
Optimiser	Adam, rate 10^{-3}	Adapts to non-stationary data
Loss	Squared error on $g(d)$	Ranking comes for free
Label window W	$8c$	Covers several cache turnovers
Replay buffer	32,768 rows	Old patterns fade out
Training step	Every 32 labels	Limits training time
Batch size	256	Stable updates
Training thread	Background, double-buffered weights	Never blocks a request
Margins	0.1 and 0.2	Network must be clearly better
Start after	W requests, 2,048 labels	No untrained decisions
Fallback window	$\max(2000, 4c)$	Enough requests to compare
Fallback tolerance	$\max(10, 1\%)$ of window	Small losses are ignored

5 Theoretical Analysis

This section gives short arguments for how the design should behave. We assume that the cache does not change which items users request.

5.1 Labels Come for Free

Every scored candidate gets exactly one label within W requests. The label depends only on when the item is requested again, not on what the cache did. This is the main difference from reinforcement learning. The cache never has to try bad moves to learn, and it never has to work out which of its past actions led to a later miss. From the network’s point of view, training is ordinary supervised regression, only with labels that arrive late and are sometimes censored.

5.2 Cost Stays Fixed

A hit does not use the network at all. A miss runs one forward pass over at most six candidates, which costs $6 \times (12 \cdot 64 + 64) \approx 5,000$ multiply and add operations. A training step, once every 32 labels, is a forward and backward pass over 256 rows, a few hundred thousand operations, and with asynchronous training it runs off the request path. The replay buffer takes about 1.7 MB, and the network with its Adam state takes about 11 kB, no matter how big the cache is. Only the per-item history grows with the cache, in the same way that ARC’s ghost lists do.

5.3 Loops Longer Than the Cache

Take a cache of c items and a workload that requests keys 1 to L in order, over and over, with $c < L$. LRU always removes the item that is needed next, so it never hits. FIFO does the same, and so does ARC. Belady’s rule removes the most recently used item, which is the one needed furthest ahead, and reaches a hit rate of $(c - 1)/(L - 1)$. A policy that keeps c keys fixed and never stores the others does better, with a hit rate of c/L , and no policy can beat it because at least $L - c$ of the keys must miss in every round. For $c = 16$ and $L = 31$ these are 50.0% and 51.6%. TIDE has both of the moves needed here: it can remove the newest item, as Belady’s rule does, and it can bypass the requested item. ARC can do neither.

5.4 When There Is Nothing to Learn

If every request is picked at random from N equally likely items, the chance of a hit is c/N for any policy that keeps the cache full, because the next request has nothing to do with what is in the cache. TIDE cannot beat ARC here, and it should not lose either. When some items are more popular than others but requests are still random, the best possible policy keeps the most popular items [1]. TIDE can only gain the difference between ARC and that best policy, which is usually small.

5.5 How Far It Can Fall Behind ARC

The fallback limits how much worse than ARC TIDE can be over a long run. Suppose the network keeps failing. After a few failures, each window in which the network is used is followed by 32 windows in which ARC decides, so at most about one window in 33 is lost. That caps the long run loss at about 3% of ARC’s hits. A workload that switches back and forth between fooling the network and suiting it perfectly could push the loss up to about a quarter of ARC’s hits, because a single good window resets the pause. Both figures assume that once ARC is back in control, the two caches quickly end up holding similar items.

6 Experimental Evaluation

6.1 Setup

Synthetic workloads. Twelve generators produce traces of 100,000 requests: uniform, Gaussian, five Zipf skews, an unbounded Zipf trace, a web trace whose hot set drifts, bursty and periodic traffic, and an adversarial loop over 31 keys. Three more produce 500,000-request traces with structure that changes over time: in *lifecycle*, items are born at random times and their popularity rises and decays; in *cycle*, 40 groups of 100 keys recur in a fixed rotating order over 30% Zipf background noise; in *phases*, each round runs one Zipf mix, a one-time scan, a loop sized to thrash a 1% cache and a second, permuted Zipf mix. Every generator is seeded, and we run two independent instances of each.

Real traces. We use the first million requests of three production traces from the public libCacheSim collection [19] (Table 5), one per domain: block storage from Microsoft Research Cambridge [12], an in-memory key-value cluster from Twitter [17], and a CDN trace from Meta. The Twitter cluster was chosen by a rule fixed in advance: the smallest of the published clusters whose gap between Belady’s rule and LRU at a 1% cache is at least two points. Two smaller clusters were skipped because no policy could improve on LRU there. Object sizes are ignored and we report object hit rate.

Table 5. Real traces (first 10^6 requests of each).

Trace	Domain	Distinct keys
MSR hm_0	Block storage	155,601
Twitter cluster 50	In-memory key-value	109,456
Meta CDN reag	Content delivery	381,431

Policies. LRU, LFU, ARC, S3-FIFO and SIEVE are our own implementations. LIRS and the learned policies LeCaR, CACHEUS and LRB are the reference implementations from libCacheSim [19], driven one request at a time with unit sizes. To check that the two sets of implementations agree, we ran our LRU,

ARC, S3-FIFO and SIEVE against libCacheSim’s on the same Zipf trace: LRU and ARC matched exactly, S3-FIFO within 0.02 points and SIEVE within 0.16 points. GL-Cache was left out because it manages space in segments of bytes and does not work with unit-size objects. As references we report Belady’s rule and the *oracle*: TIDE’s own candidates and decision rule, but scored with the true reuse distance instead of the network.

Sizes, repeats and statistics. Cache sizes are 0.1%, 1% and 10% of the distinct keys in each trace, at least 16. TIDE runs with three network seeds per trace instance; baselines are deterministic. Results are reported as the normalised hit rate $(r - r_{\text{LRU}})/(r_{\text{Belady}} - r_{\text{LRU}})$, averaged over the settings in which Belady’s rule beats LRU by more than half a point. Intervals are 95% intervals from a hierarchical bootstrap that resamples generator instances within each setting and TIDE seeds within each instance. They measure run-to-run noise for this set of workloads, not how the results would carry over to other workloads. All hit-rate experiments use the synchronous mode, which is reproducible.

6.2 Synthetic Workloads

Table 6 gives the ranking over the 43 trace and size settings (86 cells with two instances each). TIDE has the best mean normalised hit rate, and its mean hit rate is 2.19 points above ARC, 1.89 above S3-FIFO and 1.79 above CACHEUS, with intervals well clear of zero. The strongest competitor on raw hit rate is LIRS, 0.45 points behind. TIDE is at least as good as ARC in 31 of the 43 settings, and its worst setting, bursty traffic at 10%, costs 0.32 points. Figure 3 shows where the gains come from.

Table 6. Ranking on the synthetic workloads. Wins count cells in which a policy has the highest hit rate (ties count for every tied policy). Brackets are 95% bootstrap intervals.

Policy	Normalised	Hit %	Wins	TIDE – policy (points)
TIDE	0.368 [0.363, 0.372]	44.85	28	—
ARC	0.305	42.66	6	+2.19 [+2.12, +2.26]
CACHEUS	0.296	43.06	9	+1.79 [+1.70, +1.88]
S3-FIFO	0.282	42.97	18	+1.89 [+1.82, +1.96]
LIRS	0.262	44.40	10	+0.45 [+0.38, +0.52]
SIEVE	0.228	42.23	6	+2.62 [+2.54, +2.69]
LeCaR	0.053	38.86	4	+6.00 [+5.92, +6.07]
LRB	0.038	38.71	3	+6.14 [+6.07, +6.21]
LRU	0.000	38.04	3	+6.81 [+6.74, +6.88]
LFU	−1.235	35.01	6	+9.85 [+9.72, +9.97]

Table 7 shows selected settings. On the adversarial loop, recency-based policies get no hits, while TIDE reaches 48.55%, close to Belady’s 49.99%. The oracle reaches 51.60%, exactly the $c/L = 16/31$ bound of Sect. 5.3, because it

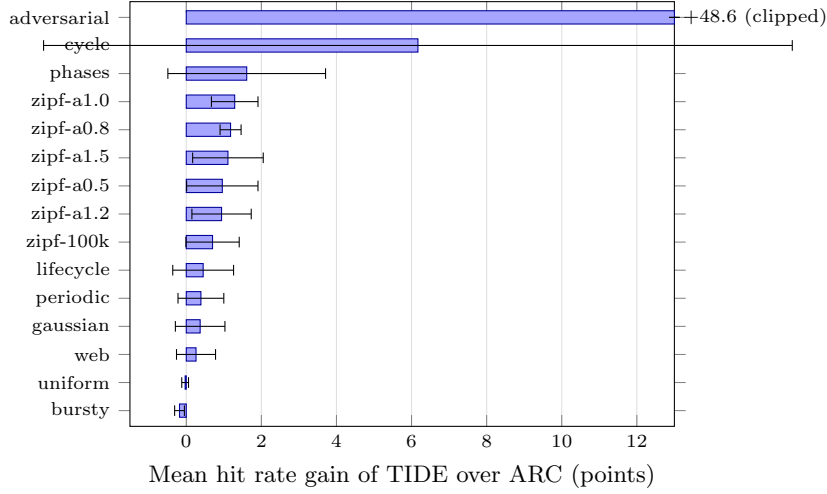


Fig. 3. Gain of TIDE over ARC per synthetic trace, averaged over three cache sizes and two instances, with 95% intervals over those six settings. The wide intervals on *cycle* and *phases* reflect how much the gain depends on cache size.

can bypass. On *cycle* at 10%, TIDE learns the period from the gap features and gains 18.4 points over ARC. On *phases* at 1% it gains 4.1 points, but with a wide spread across seeds. The table also shows that TIDE is not the best policy everywhere. LIRS, whose signal is the gap between uses, is better on *cycle* at 10% (50.36% against 42.71%) and matches TIDE on the adversarial loop, and CACHEUS is better on *phases* at 1% (46.58% against 40.87%). TIDE wins on average because it is rarely far from the best, not because it wins every setting.

Table 7. Hit rate (%) in selected synthetic settings, mean over two instances; TIDE with a 95% interval over instances and seeds. On the adversarial loop every size rounds to $c = 16$.

Trace	Size	LRU	ARC	S3-FIFO	LIRS	CACHEUS	TIDE	Oracle	Belady
adversarial	–	0.00	0.00	0.00	48.38	0.00	48.55 ± 1.94	51.60	49.99
cycle	1%	15.03	22.05	21.92	22.22	21.26	21.93 ± 0.25	22.25	23.72
cycle	10%	20.87	24.29	24.15	50.36	41.57	42.71 ± 0.88	43.71	53.21
phases	1%	32.57	36.77	36.50	36.79	46.58	40.87 ± 3.60	53.12	60.44
phases	10%	69.42	69.26	69.61	68.45	69.13	69.01 ± 0.07	70.21	74.50
zipf-100k	1%	59.43	67.28	67.35	67.45	67.48	67.57 ± 0.03	68.19	71.26
uniform	1%	1.01	1.00	1.01	1.03	0.98	1.02 ± 0.04	8.56	13.17
bursty	10%	85.52	85.51	85.50	85.51	85.47	85.19 ± 0.24	87.04	89.36

LeCaR and LRB stay close to LRU on the synthetic traces. For LRB this is expected: it needs a long request history before its model is trained, and these traces are short. On a three-million-request prefix of the Twitter trace, which we ran as a check, LRB was 2 to 5 points above LRU throughout.

6.3 Real Traces

Table 8 shows the results on the real traces, and here the picture changes. TIDE is at least as good as ARC in only one of the nine settings, and on average it trails ARC by 0.39 points $[-0.43, -0.34]$. It still beats S3-FIFO by 0.41 points $[+0.36, +0.45]$, CACHEUS by 1.61, LRB by 2.24 and LIRS by 4.10, and it has the second-best mean normalised hit rate (0.113, against 0.179 for ARC). Its worst setting is the Meta CDN trace at 10%, 1.13 points below ARC.

Table 8. Hit rate (%) on the real traces (first 10^6 requests). Active is the share of requests on which TIDE’s network, rather than ARC, made the decision.

Trace	Size	c	ARC	S3-FIFO	LIRS	CACHEUS	LRB	TIDE	Oracle	Belady	Active
MSR	0.1%	156	54.77	54.74	53.97	51.84	52.77	54.65	57.02	59.64	61%
	1%	1,556	64.53	63.95	63.96	62.41	63.41	64.54	66.56	69.91	53%
	10%	15,560	74.08	74.22	73.85	72.52	71.68	73.76	74.25	79.20	35%
Twitter	0.1%	109	24.74	16.13	1.30	24.73	11.11	24.50	26.44	31.72	70%
	1%	1,095	33.41	35.07	24.40	26.76	30.68	33.03	36.56	51.08	4%
	10%	10,946	65.62	65.69	65.31	63.30	65.67	64.83	66.42	81.03	23%
Meta	0.1%	381	50.08	49.50	46.48	49.02	49.40	49.95	51.92	55.30	16%
	1%	3,814	54.42	55.02	52.97	53.23	53.60	54.07	55.57	59.91	8%
	10%	38,143	58.92	59.15	58.01	58.78	58.66	57.79	59.35	61.86	31%

Two observations explain this. First, the oracle is on average only 1.5 points above ARC on these traces, so even perfect predictions within TIDE’s candidate set would gain little: the structure that TIDE exploits on the synthetic traces, loops and periodic returns, is rare here. Second, the network was in control for only 4 to 70% of requests. Whenever it fell behind the shadow ARC, the fallback handed control back, which is why the losses stay near a point rather than growing. The fallback therefore did its job, but the network did not learn a better rule than ARC’s within a million requests. (On the Twitter trace at 10%, LRU happens to beat every other policy, including ARC.)

6.4 Speed and Asynchronous Training

Table 9 reports the latency of every request, hits included, on the first 500,000 requests of each real trace at the 1% cache size, measured one process at a time. Compiling the hot path lowers the 99th percentile from 133–219 μ s to 87–112 μ s, and moving training to the background thread lowers it further to 18–26 μ s. Before, the tail was set by the requests that triggered a training step. The median, 2.9–9.4 μ s, is dominated by Python bookkeeping around the network: the ARC lists, the shadow ARC and the label queue. Throughput is 1.2 to 2.2 times that of the NumPy version, but still 10 to 16 times below ARC.

Asynchronous training means decisions use weights that are slightly out of date. Table 10 measures the cost at the 1% size over three seeds. On four of five traces the hit rate changes by at most 0.17 points. On *phases* it drops by

Table 9. Per-request latency and throughput at the 1% cache size.

Trace	Implementation	Req/s	p50 (μ s)	p99 (μ s)	p99.9 (μ s)
MSR	ARC	2,487k	0.16	1.0	3.1
	TIDE, NumPy	128k	2.56	133.5	183.3
	TIDE, Numba, inline training	150k	2.49	87.3	134.3
	TIDE, Numba, async training	155k	2.90	25.8	64.1
Twitter	ARC	1,099k	0.79	1.8	4.3
	TIDE, NumPy	48k	14.15	219.3	410.8
	TIDE, Numba, inline training	83k	9.62	112.2	125.2
	TIDE, Numba, async training	108k	9.43	20.8	58.2
Meta	ARC	1,649k	0.28	1.0	5.0
	TIDE, NumPy	81k	7.16	186.8	221.9
	TIDE, Numba, inline training	122k	4.99	95.9	110.6
	TIDE, Numba, async training	145k	5.12	17.5	62.7

3.10 points, although the trainer completes as many steps as in synchronous mode. This trace switches regime abruptly, and learning the new regime quickly depends on fresh weights at exactly those moments.

Table 10. Hit-rate cost of asynchronous training (1% cache size, mean over three seeds).

Trace	Sync (%)	Async (%)	Change	Steps done
cycle	21.90	22.07	+0.17	99%
phases	40.72	37.63	-3.10	104%
MSR	64.54	64.44	-0.10	100%
Twitter	33.03	33.04	+0.01	100%
Meta	54.07	54.04	-0.02	100%

6.5 Discussion

The results support the analysis where the analysis is specific. On loops, TIDE finds the moves ARC lacks and approaches Belady’s rule. On uniform traffic, nothing can be learned and TIDE ties ARC. The fallback keeps losses near a point in every setting. The results also show the limits of the design. Its gains depend on structure that recurs within the label window, loops and periodic returns, and that structure was common in our synthetic workloads but rare in the real traces we tested, where the oracle itself is barely above ARC. Where such structure exists, simpler signals can capture it as well or better: LIRS on *cycle*, CACHEUS on *phases*. TIDE’s advantage on the synthetic suite comes

from being close to the best on most workloads at once, not from dominating any one of them.

7 Limitations

The experiments were run in a reduced configuration to fit a ten-minute budget: two instances per generator, three network seeds, 500,000-request trend traces and million-request prefixes of the real traces. Longer traces could favour both TIDE and LRB, which need long histories, and the real-trace intervals reflect only seed noise, since each trace has one instance. Three real traces cannot represent all production workloads, and object sizes were ignored. The loss bound assumes that TIDE and the shadow quickly hold similar items once ARC is back in control, which is only roughly true. Items that return after more than W requests all get the same censored label, so the network cannot tell them apart. The candidates the network sees depend on its own earlier choices, so what it trains on may drift from what it is asked to judge. All settings are fixed in advance and not tuned per workload. Finally, even compiled, TIDE handles 10 to 16 times fewer requests per second than ARC, because much of its per-request work is still interpreted Python; a native implementation of the whole cache would be needed before production use.

8 Conclusion

TIDE keeps ARC’s choice of which list to shrink and hands the choice of which item to remove to a compact neural network, with the option of not caching a new item at all. The network regresses a scaled reuse distance, the quantity Belady’s rule ranks by, and is trained continually by backpropagation from labels that the request stream produces on its own, handled as delayed and censored observations, replayed through an experience buffer and applied by a background thread. On fifteen synthetic workloads it has the best mean normalised hit rate among eleven policies, including three learned ones, and it gains up to 48.6 points where recency is misleading. On three real traces it does not beat ARC, trailing it by 0.39 points, although it beats S3-FIFO and every learned baseline, and its fallback keeps every loss near a point. Asynchronous training brings the 99th-percentile latency down to about 20 μ s. Longer real traces, workloads with recurring structure, and a native implementation are the natural next steps.

References

1. Aho, A.V., Denning, P.J., Ullman, J.D.: Principles of optimal page replacement. *Journal of the ACM* **18**(1), 80–93 (1971)
2. Belady, L.A.: A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal* **5**(2), 78–101 (1966)

3. He, K., Zhang, X., Ren, S., Sun, J.: Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In: Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 1026–1034 (2015)
4. Jain, A., Lin, C.: Back to the future: Leveraging Belady’s algorithm for improved cache replacement. In: Proceedings of the 43rd International Symposium on Computer Architecture (ISCA), pp. 78–89 (2016)
5. Jiang, S., Zhang, X.: LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance. In: Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, pp. 31–42 (2002)
6. Johnson, T., Shasha, D.: 2Q: A low overhead high performance buffer management replacement algorithm. In: Proceedings of the 20th International Conference on Very Large Data Bases, pp. 439–450 (1994)
7. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: Proceedings of the 3rd International Conference on Learning Representations (ICLR) (2015)
8. Lam, S.K., Pitrou, A., Seibert, S.: Numba: A LLVM-based Python JIT compiler. In: Proceedings of the 2nd Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC), pp. 1–6 (2015)
9. Lin, L.J.: Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning* **8**(3–4), 293–321 (1992)
10. Liu, E.Z., Hashemi, M., Swersky, K., Ranganathan, P., Ahn, J.: An imitation learning approach for cache replacement. In: Proceedings of the 37th International Conference on Machine Learning (ICML), pp. 6237–6247 (2020)
11. Megiddo, N., Modha, D.S.: ARC: A self-tuning, low overhead replacement cache. In: Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST), pp. 115–130 (2003)
12. Narayanan, D., Donnelly, A., Rowstron, A.: Write off-loading: Practical power management for enterprise storage. In: Proceedings of the 6th USENIX Conference on File and Storage Technologies (FAST), pp. 253–267 (2008)
13. Rodriguez, L.V., Yusuf, F., Lyons, S., Paz, E., Rangaswami, R., Liu, J., Zhao, M., Narasimhan, G.: Learning cache replacement with CACHEUS. In: Proceedings of the 19th USENIX Conference on File and Storage Technologies (FAST), pp. 341–354 (2021)
14. Song, Z., Berger, D.S., Li, K., Lloyd, W.: Learning relaxed Belady for content distribution network caching. In: Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI), pp. 529–544 (2020)
15. Song, Z., Chen, K., Sarda, N., Altınbüken, D., Brevdo, E., Coleman, J., Ju, X., Jurczyk, P., Schooler, R., Gummadi, R.: HALP: Heuristic aided learned preference eviction policy for YouTube content delivery network. In: Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI), pp. 1149–1163 (2023)
16. Vietri, G., Rodriguez, L.V., Martinez, W.A., Lyons, S., Liu, J., Rangaswami, R., Zhao, M., Narasimhan, G.: Driving cache replacement with ML-based LeCaR. In: Proceedings of the 10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage) (2018)
17. Yang, J., Yue, Y., Rashmi, K.V.: A large scale analysis of hundreds of in-memory cache clusters at Twitter. In: Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pp. 191–208 (2020)
18. Yang, J., Zhang, Y., Qiu, Z., Yue, Y., Vinayak, R.: FIFO queues are all you need for cache eviction. In: Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP), pp. 130–149 (2023)

19. Yang, J., et al.: libCacheSim: A high-performance library for building cache simulators. <https://github.com/1a1a11a/libCacheSim> (2023). Traces from the public libCacheSim dataset collection
20. Zhang, Y., Yang, J., Yue, Y., Vigfusson, Y., Rashmi, K.V.: SIEVE is simpler than LRU: An efficient turn-key eviction algorithm for web caches. In: Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI) (2024)